

第 9 章

图像梯度

图像梯度计算的是图像变化的速度。对于图像的边缘部分，其灰度值变化较大，梯度值也较大；相反，对于图像中比较平滑的部分，其灰度值变化较小，相应的梯度值也较小。一般情况下，图像梯度计算的是图像的边缘信息。

严格来讲，图像梯度计算需要求导数，但是图像梯度一般通过计算像素值的差来得到梯度的近似值（近似导数值）。

例如，图 9-1 中的左右两幅图分别描述了图像的水平边界和垂直边界。

针对左图，通过垂直方向的线条 A 和线条 B 的位置，可以计算图像水平方向的边界：

- 对于线条 A 和线条 B，其右侧像素值与左侧像素值的差值不为零，因此是边界。
- 对于其余列，其右侧像素值与左侧像素值的差值均为零，因此不是边界。

针对右图，通过水平方向的线条 A 和线条 B 的位置，可以计算图像垂直方向的边界：

- 对于线条 A 和线条 B，其下侧像素值与上侧像素值的差值不为零，因此是边界。
- 对于其余行，其下侧像素值与上侧像素值的差值均为零，因此不是边界。

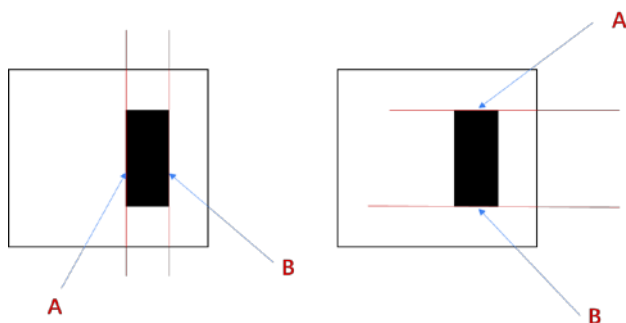


图 9-1 图像边界示意图

将上述运算关系进一步优化，可以得到更复杂的边缘信息。本章将关注 Sobel 算子、Scharr 算子和 Laplacian 算子的使用。

9.1 Sobel 理论基础

Sobel 算子是一种离散的微分算子，该算子结合了高斯平滑和微分求导运算。该算子利用局部差分寻找边缘，计算所得的是一个梯度的近似值。



Sobel 算子如图 9-2 所示。

-1	0	1	-1	-2	-1
-2	0	2	0	0	0
-1	0	1	1	2	1

图 9-2 Sobel 算子示例

需要说明的是，滤波器通常是指由一幅图像根据像素点(x, y)临近的区域计算得到另外一幅新图像的算法。因此，滤波器是由邻域及预定义的操作构成的。滤波器规定了滤波时所采用的形状以及该区域内像素值的组成规律。滤波器也被称为“掩模”、“核”、“模板”、“窗口”、“算子”等。一般信号领域将其称为“滤波器”，数学领域将其称为“核”。本章中出现的滤波器多数为“线性滤波器”，也就是说，滤波的目标像素点的值等于原始像素值及其周围像素值的加权和。这种基于线性核的滤波，就是我们所熟悉的卷积。在本章中，为了方便说明，直接使用“算子”来表示各种算子所使用的滤波器。例如，本章中所说的“Sobel 算子”通常是指 Sobel 滤波器。

假定有原始图像 src，下面对 Sobel 算子的计算进行讨论。

1. 计算水平方向偏导数的近似值

将 Sobel 算子与原始图像 src 进行卷积计算，可以计算水平方向上的像素值变化情况。例如，当 Sobel 算子的大小为 3×3 时，水平方向偏导数 G_x 的计算方式为：

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \cdot \text{src}$$

上式中，src 是原始图像，假设其中有 9 个像素点，如图 9-3 所示。

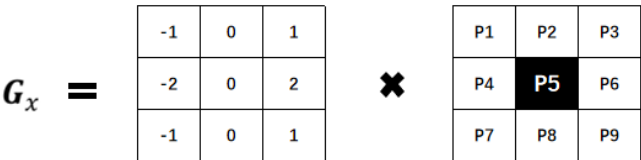


图 9-3 水平方向偏导数计算说明

如果要计算像素点 P5 的水平方向偏导数 $P5_x$ ，则需要利用 Sobel 算子及 P5 邻域点，所使用的公式为：

$$P5_x = (P3-P1) + 2 \cdot (P6-P4) + (P9-P7)$$

即用像素点 P5 右侧像素点的像素值减去其左侧像素点的像素值。其中，中间像素点(P4 和 P6) 距离像素点 P5 较近，其像素值差值的权重为 2；其余差值的权重为 1。

2. 计算垂直方向偏导数的近似值

将 Sobel 算子与原始图像 src 进行卷积计算，可以计算垂直方向上的变化情况。例如，当 Sobel 算子的大小为 3×3 时，垂直方向偏导数 G_y 的计算方式为：

$$G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \cdot \text{src}$$

上式中，src 是原始图像，假设其中有 9 个像素点，如图 9-4 所示。

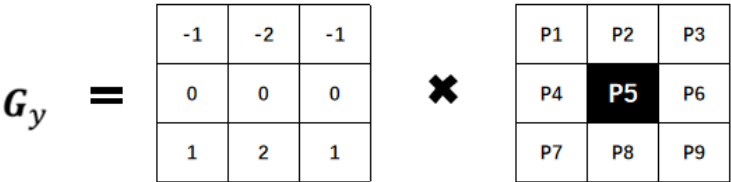


图 9-4 垂直方向偏导数计算说明

如果要计算像素点 P5 的垂直方向偏导数 P5_y，则需要利用 Sobel 算子及 P5 邻域点，所使用的公式为：

$$P5_y = (P7-P1) + 2 \cdot (P8-P2) + (P9-P3)$$

式中，使用像素点 P5 下一行像素点的像素值减去上一行像素点的像素值。其中，中间像素点（P2 和 P8）距离像素点 P5 较近，其像素值差值的权重为 2；其余差值的权重为 1。

9.2 Sobel 算子及函数使用

在 OpenCV 内，使用函数 cv2.Sobel()实现 Sobel 算子运算，其语法形式为：

dst = cv2.Sobel(src, ddepth, dx, dy[, ksize[, scale[, delta[, borderType]]]])

式中：

- dst 代表目标图像。
- src 代表原始图像。
- ddepth 代表输出图像的深度。其具体对应关系如表 9-1 所示。

表 9-1 ddepth 值

输入图像深度（src.depth()）	输出图像深度（ddepth）
cv2.CV_8U	-1/cv2.CV_16S/cv2.CV_32F/cv2.CV_64F
cv2.CV_16U/cv2.CV_16S	-1/cv2.CV_32F/cv2.CV_64F
cv2.CV_32F	-1/cv2.CV_32F/cv2.CV_64F
cv2.CV_64F	-1/cv2.CV_64F

- dx 代表 x 方向上的求导阶数。
- dy 代表 y 方向上的求导阶数。
- ksize 代表 Sobel 核的大小。该值为-1 时，则会使用 Scharr 算子进行运算。
- scale 代表计算导数值时所采用的缩放因子，默认情况下该值是 1，是没有缩放的。
- delta 代表加在目标图像 dst 上的值，该值是可选的，默认为 0。
- borderType 代表边界样式。该参数的具体类型及值如表 9-2 所示。



表 9-2 borderType 类型及值

类型	具体值
cv2.BORDER_CONSTANT	iiii abcdefgh iiii, 特定的 i
cv2.BORDER_REPLICATE	aaaaa abcdefgh hhhhhhh
cv2.BORDER_REFLECT	fedcba abcdefgh hgfedcb
cv2.BORDER_WRAP	cdefgh abcdefgh abcdefg
cv2.BORDER_REFLECT_101	gfedcb abcdefgh gfedcba
cv2.BORDER_TRANSPARENT	uvwxyz abcdefgh ijklmno
cv2.BORDER_REFLECT101	和 BORDER_REFLECT_101 一致
cv2.BORDER_DEFAULT	和 BORDER_REFLECT_101 一致
cv2.BORDER_ISOLATED	不考虑 ROI (Region of Interest, 感兴趣区域) 以外的区域

9.2.1 参数ddepth

在函数 cv2.Sobel()的语法中规定，可以将函数 cv2.Sobel()内 ddepth 参数的值设置为-1，让处理结果与原始图像保持一致。但是，如果直接将参数 ddepth 的值设置为-1，在计算时得到的结果可能是错误的。

在实际操作中，计算梯度值可能会出现负数。如果处理的图像是 8 位图类型，则在 ddepth 的参数值为-1 时，意味着指定运算结果也是 8 位图类型，那么所有负数会自动截断为 0，发生信息丢失。为了避免信息丢失，在计算时要先使用更高的数据类型 cv2.CV_64F，再通过取绝对值将其映射为 cv2.CV_8U（8 位图）类型。所以，通常要将函数 cv2.Sobel()内参数 ddepth 的值设置为“cv2.CV_64F”。

下面对参数 ddepth 值的设定做一个简要的说明。

例如，图 9-5 中的原始图像（左图）是一幅二值图像，图中黑色部分的像素值为 0，白色部分的像素值为 1。在计算 A 线条所在位置和 B 线条所在位置的近似偏导数时：

- 针对 A 线条所在列，右侧像素值减去左侧像素值所得近似偏导数的值为-1。
- 针对 B 线条所在列，右侧像素值减去左侧像素值所得近似偏导数的值为 1。

针对上述偏导数结果进行不同方式的处理，可能会得到不同的结果，例如：

- 直接计算。此时，A 线条位置的值为负数，B 线条位置的值为正数。在显示时，由于上述负值不在 8 位图范围内，因此要做额外处理。将 A 线条处的负数偏导数处理为 0，B 线条处的正数偏导数保持不变。在这种情况下，显示结果如图 9-5 中间的图所示。
- 计算绝对值。此时，A 线条处的负数偏导数被处理正数，B 线条处的正数偏导数保持不变。在显示时，由于上述值在 8 位图的表示范围内，因此不再对上述值进行处理，此时，显示结果如图 9-5 中的右图所示。

上述问题在计算垂直方向的近似偏导数时同样存在。例如，图 9-6 中的原始图像（左图）是一幅二值图像，图中黑色部分的像素值为 0，白色部分的像素值为 1。计算 A 线条所在位置和 B 线条所在位置的近似偏导数时：

- 针对 A 线条所在行，下方像素值减去上方像素值所得近似偏导数为-1。
- 针对 B 线条所在行，下方像素值减去上方像素值所得近似偏导数为 1。

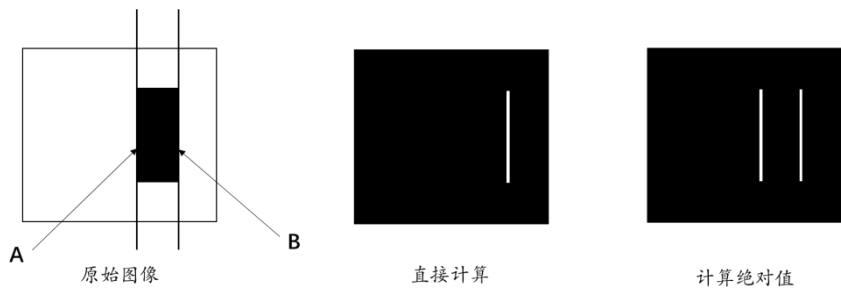


图 9-5 显示水平方向的近似偏导数

针对上述偏导数结果进行不同方式的处理，可能会得到不同的结果，例如：

- 直接计算。此时，A 线条位置的值为负数，B 线条位置的值为正数。在显示时，上述值不在 8 位图的表示范围内，因此需要进行额外处理。将 A 线条处的负数偏导数处理为 0，B 线条处的正数偏导数保持不变。此时，显示结果如图 9-6 中间的图所示。
- 计算绝对值。此时，A 线条处的负数偏导数被处理正数，B 线条处的正数偏导数保持不变。在显示时，由于上述值在 8 位图的表示范围内，因此不再对上述值进行处理，此时，显示结果如图 9-6 中的右图所示。

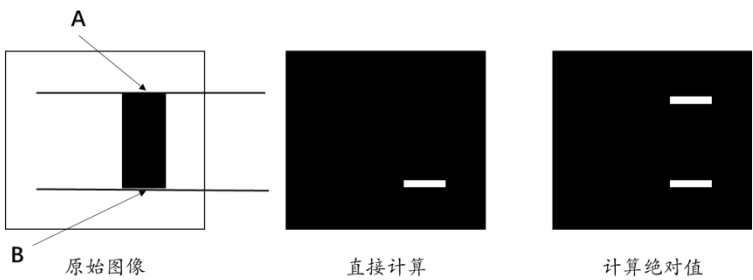


图 9-6 显示垂直方向的近似偏导数

经过上述分析可知，为了让偏导数正确地显示出来，需要将值为负数的近似偏导数转换为正数。即，要将偏导数取绝对值，以保证偏导数总能正确地显示出来。

例如，图 9-7 描述了如何计算偏导数。

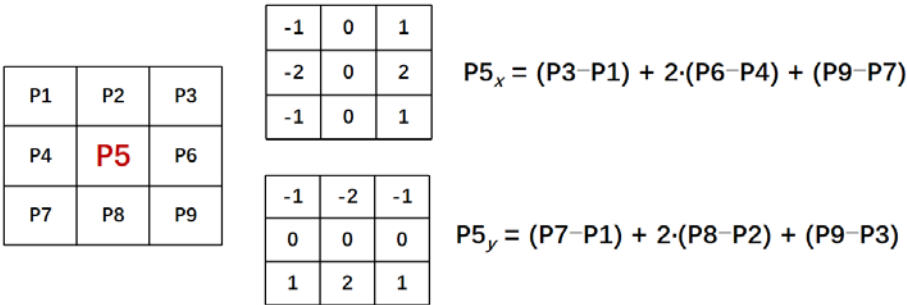


图 9-7 偏导数计算

为了得到结果为正数的偏导数，需要对图 9-7 中计算的偏导数取绝对值，如下：

$$|P5_x| = |(P3-P1) + 2 \cdot (P6-P4) + (P9-P7)|$$

$$|P5_y| = |(P7-P1) + 2 \cdot (P8-P2) + (P9-P3)|$$

当然，在需要时，还可以进行如下处理：

$$P5Sobel = |P5_x| + |P5_y| = |(P3-P1) + 2 \cdot (P6-P4) + (P9-P7)| + |(P7-P1) + 2 \cdot (P8-P2) + (P9-P3)|$$

经过以上分析，我们得知：在实际操作中，计算梯度值可能会出现负数。通常处理的图像是 8 位图类型，如果结果也是该类型，那么所有负数会自动截断为 0，发生信息丢失。所以，为了避免信息丢失，我们在计算时使用更高的数据类型 `cv2.CV_64F`，再通过取绝对值将其映射为 `cv2.CV_8U`（8 位图）类型。

在 OpenCV 中，使用函数 `cv2.convertScaleAbs()` 对参数取绝对值，该函数的语法格式为：

```
dst = cv2.convertScaleAbs( src [, alpha[, beta]] )
```

上式中：

- `dst` 代表处理结果。
- `src` 代表原始图像。
- `alpha` 代表调节系数，该值是可选值，默认为 1。
- `beta` 代表调节亮度值，该值是默认值，默认为 0。

这里，该函数的作用是将原始图像 `src` 转换为 256 色位图，其可以表示为：

```
dst=saturate(src*alpha+beta)
```

式中，`saturate()` 表示计算结果的最大值是饱和值，例如当 “`src*alpha+beta`” 的值超过 255 时，其取值为 255。

【例 9.1】 使用函数 `cv2.convertScaleAbs()` 对一个随机数组取绝对值。

根据题目要求，编写代码如下：

```
import cv2
import numpy as np
img=np.random.randint(-256,256,size=[4,5],dtype=np.int16)
rst=cv2.convertScaleAbs(img)
print("img=\n",img)
print("rst=\n",rst)
```

运行程序，结果为：

```
img=
[[-138 122 -87 -54 185]
 [ 227 -221 74 48 -11]
 [ 121 217 45 -237 -140]
 [-87 71 -74 -151 -188]]
rst=
[[138 122 87 54 185]
 [227 221 74 48 11]
 [121 217 45 237 140]
 [ 87 71 74 151 188]]
```

9.2.2 方向

在函数 `cv2.Sobel()` 中, 参数 `dx` 表示 x 轴方向的求导阶数, 参数 `dy` 表示 y 轴方向的求导阶数。参数 `dx` 和 `dy` 通常为 0 或者 1, 最大值为 2。如果是 0, 表示在该方向上没有求导。当然, 参数 `dx` 和参数 `dy` 的值不能同时为 0。

参数 `dx` 和参数 `dy` 可以有多种形式的组合, 主要包含:

- 计算 x 方向边缘 (梯度): `dx=1, dy=0`。
- 计算 y 方向边缘 (梯度): `dx=0, dy=1`。
- 参数 `dx` 与参数 `dy` 的值均为 1: `dx=1, dy=1`。
- 计算 x 方向和 y 方向的边缘叠加: 通过组合方式实现。

下面分别对上述情况进行简要说明。

1. 计算 x 方向边缘 (梯度): `dx=1, dy=0`

如果想只计算 x 方向 (水平方向) 的边缘, 需要将函数 `cv2.Sobel()` 的参数 `dx` 和 `dy` 的值设置为 “`dx=1, dy=0`”。当然, 也可以设置为 “`dx=2, dy=0`”。此时, 会仅仅获取水平方向的边缘信息, 此时的语法格式为:

```
dst = cv2.Sobel( src , ddepth , 1 , 0 )
```

使用该语句获取边缘图的示例如图 9-8 所示, 其中左图为原始图像, 右图为获取的边缘图。

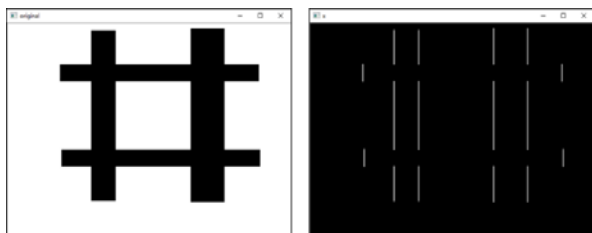


图 9-8 获取水平方向的边缘

2. 计算 y 方向边缘 (梯度): `dx=0, dy=1`

如果想只计算 y 方向 (垂直方向) 的边缘, 需要将函数 `cv2.Sobel()` 的参数 `dx` 和 `dy` 的值设置为 “`dx=0, dy=1`”。当然, 也可以设置为 “`dx=0, dy=2`”。此时, 会仅仅获取垂直方向的边缘信息, 此时的语法格式为:

```
dst = cv2.Sobel( src , ddepth , 0 , 1 )
```

使用该语句获取边缘图的示例如图 9-9 所示, 其中左图为原始图像, 右图为获取的边缘图。

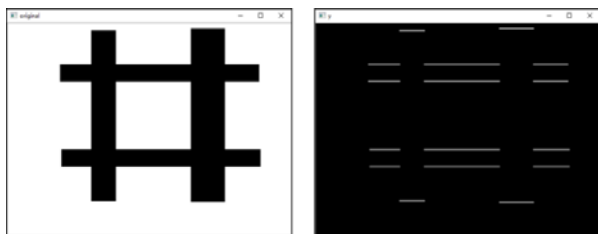


图 9-9 获取垂直方向的边缘

3. 参数dx与参数dy的值均为 1: dx=1,dy=1

可以将函数 `cv2.Sobel()` 的参数 `dx` 和 `dy` 的值设置为“`dx=1, dy=1`”，也可以设置为“`dx=2, dy=2`”，或者两个参数都不为零的其他情况。此时，会获取两个方向的边缘信息，此时的语法格式为：

```
dst = cv2.Sobel( src , ddepth , 1 , 1 )
```

使用该语句获取边缘图的示例如图 9-10 所示，其中左图为原始图像，右图为获取的边缘图，仔细观察可以看到图中仅有若干个微小白点，每个点的大小为一个像素。

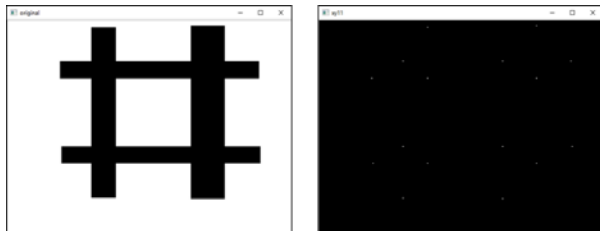


图 9-10 dx=1, dy=1 时获取的边缘图

4. 计算x方向和y方向的边缘叠加

如果想获取 `x` 方向和 `y` 方向的边缘叠加，需要分别获取水平方向、垂直方向两个方向的边缘图，然后将二者相加。此时的语法格式为：

```
dx= cv2.Sobel( src , ddepth , 1 , 0 )
dy= cv2.Sobel( src , ddepth , 0 , 1 )
dst=cv2.addWeighted( src1 , alpha , src2 , beta , gamma )
```

使用上述语句获取边缘图的示意图如图 9-11 所示，其中左图为原始图像，右图为获取的边缘图。

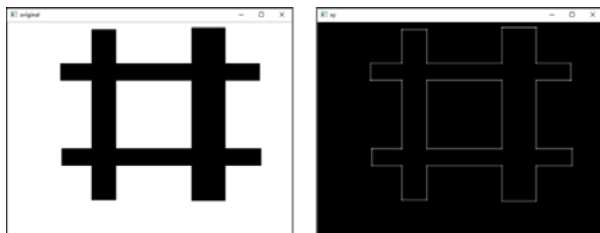


图 9-11 获取水平方向和垂直方向的边缘叠加

9.2.3 实例

本节将通过实例来介绍如何使用函数 `cv2.Sobel()` 获取图像边缘信息。

【例 9.2】 使用函数 `cv2.Sobel()` 获取图像水平方向的边缘信息。

在本例中，将参数 `ddepth` 的值设置为-1，参数 `dx` 和 `dy` 的值设置为“`dx=1, dy=0`”。

根据题目要求及分析，设计程序如下：

```
import cv2
o = cv2.imread('Sobel4.bmp',cv2.IMREAD_GRAYSCALE)
```



```
Sobelx = cv2.Sobel(o, -1, 1, 0)
cv2.imshow("original", o)
cv2.imshow("x", Sobelx)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行程序，结果如图 9-12 所示。

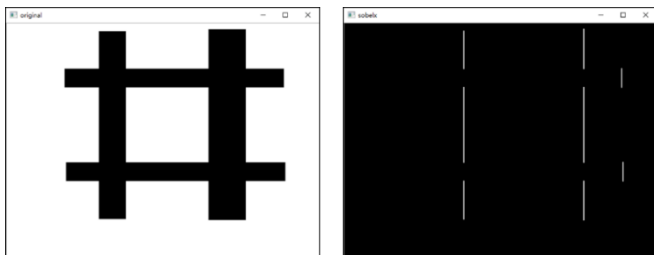


图 9-12 【例 9.2】程序的运行结果

【分析】

从程序可以看出，当参数 `ddepth` 的值为 `-1` 时，只得到了图中黑色框的右边界。这是因为，左边界在运算时得到了负值，其在显示时被调整为 `0`，所以没有显示出来。要想获取左边界的值（将其显示出来），必须将参数 `ddepth` 的值设置为更大范围的数据结构类型，并将其映射到 8 位图像内。

【例 9.3】 使用函数 `cv2.Sobel()` 获取图像水平方向的完整边缘信息。

在本例中，将参数 `ddepth` 的值设置为 `cv2.CV_64F`，并使用函数 `cv2.convertScaleAbs()` 对 `cv2.Sobel()` 的计算结果取绝对值。

根据题目要求及分析，设计程序如下：

```
import cv2
o = cv2.imread('Sobel4.bmp', cv2.IMREAD_GRAYSCALE)
Sobelx = cv2.Sobel(o, cv2.CV_64F, 1, 0)
Sobelx = cv2.convertScaleAbs(Sobelx)
cv2.imshow("original", o)
cv2.imshow("x", Sobelx)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行程序，结果如图 9-13 所示。

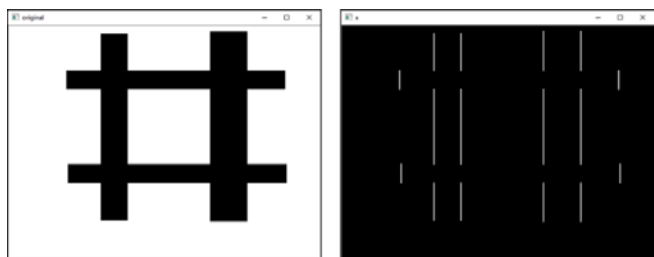


图 9-13 【例 9.3】程序的运行结果

【分析】

从程序可以看出，将函数 `cv2.Sobel()` 内参数 `ddepth` 的值设置为“`cv2.CV_64F`”，参数 `dx` 和 `dy` 的值设置为“`dx=1, dy=0`”后，执行该函数，再对该函数的结果计算绝对值，可以获取图像在水平方向的完整边缘信息。

【例 9.4】 使用函数 `cv2.Sobel()` 获取图像垂直方向的边缘信息。

在本例中，将参数 `ddepth` 的值设置为 `cv2.CV_64F`，并使用函数 `cv2.convertScaleAbs()` 对 `cv2.Sobel()` 的计算结果取绝对值。

根据题目要求及分析，设计程序如下：

```
import cv2
o = cv2.imread('Sobel4.bmp', cv2.IMREAD_GRAYSCALE)
Sobely = cv2.Sobel(o, cv2.CV_64F, 0, 1)
Sobely = cv2.convertScaleAbs(Sobely)
cv2.imshow("original", o)
cv2.imshow("y", Sobely)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行程序，结果如图 9-14 所示。

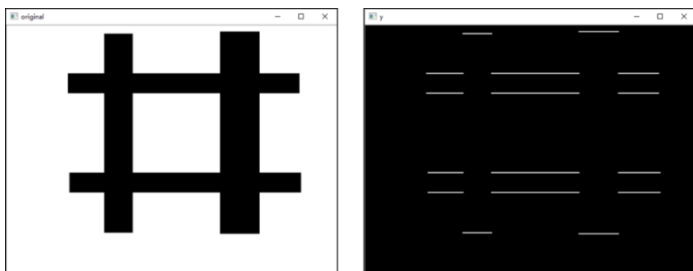


图 9-14 【例 9.4】程序的运行结果

【分析】

从程序可以看出，将参数 `ddepth` 的值设置为“`cv2.CV_64F`”，参数 `dx` 和 `dy` 的值设置为“`dx=0, dy=1`”的情况下，使用函数 `cv2.convertScaleAbs()` 对函数 `cv2.Sobel()` 的计算结果取绝对值，可以获取图像在垂直方向的完整边缘信息。

【例 9.5】 当参数 `dx` 和 `dy` 的值为“`dx=1, dy=1`”时，查看函数 `cv2.Sobel()` 的执行效果。

根据题目要求，设计程序如下：

```
import cv2
o = cv2.imread('Sobel4.bmp', cv2.IMREAD_GRAYSCALE)
Sobelxy = cv2.Sobel(o, cv2.CV_64F, 1, 1)
Sobelxy = cv2.convertScaleAbs(Sobelxy)
cv2.imshow("original", o)
cv2.imshow("xy", Sobelxy)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行程序，结果如图 9-15 所示。

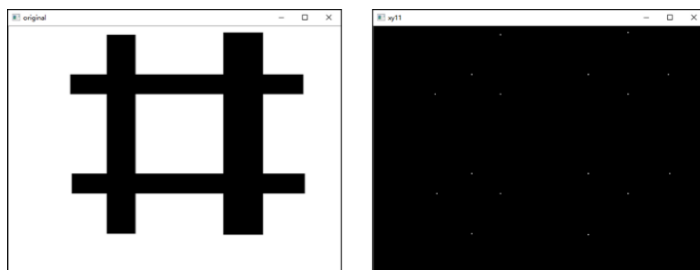


图 9-15 【例 9.5】程序的运行结果

【例 9.6】 计算函数 `cv2.Sobel()` 在水平、垂直两个方向叠加的边缘信息。

根据题目要求，设计程序如下：

```
import cv2
o = cv2.imread('Sobel4.bmp', cv2.IMREAD_GRAYSCALE)
Sobelx = cv2.Sobel(o, cv2.CV_64F, 1, 0)
Sobely = cv2.Sobel(o, cv2.CV_64F, 0, 1)
Sobelx = cv2.convertScaleAbs(Sobelx)
Sobely = cv2.convertScaleAbs(Sobely)
Sobelxy = cv2.addWeighted(Sobelx, 0.5, Sobely, 0.5, 0)
cv2.imshow("original", o)
cv2.imshow("xy", Sobelxy)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行程序，结果如图 9-16 所示。

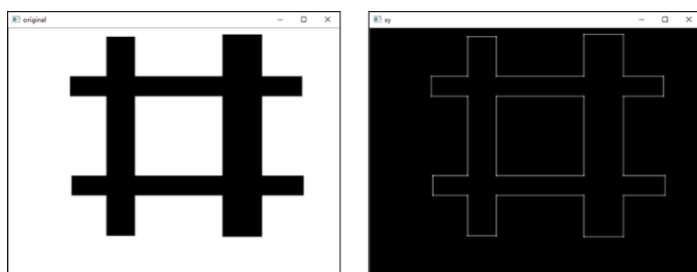


图 9-16 【例 9.6】程序的运行结果

【分析】

从程序可以看出，本例中首先分别计算 x 方向的边缘、 y 方向的边缘，接下来使用函数 `cv2.addWeighted()` 对两个方向的边缘进行叠加。在最终的叠加边缘结果中，同时显示两个方向的边缘信息。

【例 9.7】 使用不同方式处理图像在两个方向的边缘信息。

在本例中，分别使用两种不同的方式获取边缘信息。

- 方式 1：分别使用“ $dx=1, dy=0$ ”和“ $dx=0, dy=1$ ”计算图像在水平方向和垂直方向的边缘信息，然后将二者相加，构成两个方向的边缘信息。



- 方式 2：将参数 dx 和 dy 的值设为 “dx=1, dy=1”，获取图像在两个方向的梯度。

根据题目要求，设计程序如下：

```
import cv2
o = cv2.imread('lena.bmp', cv2.IMREAD_GRAYSCALE)
Sobelx = cv2.Sobel(o, cv2.CV_64F, 1, 0)
Sobely = cv2.Sobel(o, cv2.CV_64F, 0, 1)
Sobelx = cv2.convertScaleAbs(Sobelx)
Sobely = cv2.convertScaleAbs(Sobely)
Sobelxy = cv2.addWeighted(Sobelx, 0.5, Sobely, 0.5, 0)
Sobelxy11 = cv2.Sobel(o, cv2.CV_64F, 1, 1)
Sobelxy11 = cv2.convertScaleAbs(Sobelxy11)
cv2.imshow("original", o)
cv2.imshow("xy", Sobelxy)
cv2.imshow("xy11", Sobelxy11)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行程序，结果如图 9-17 所示，其中左图为原始图像，中间的图为方式 1 对应的图像，右图为方式 2 对应的图像。

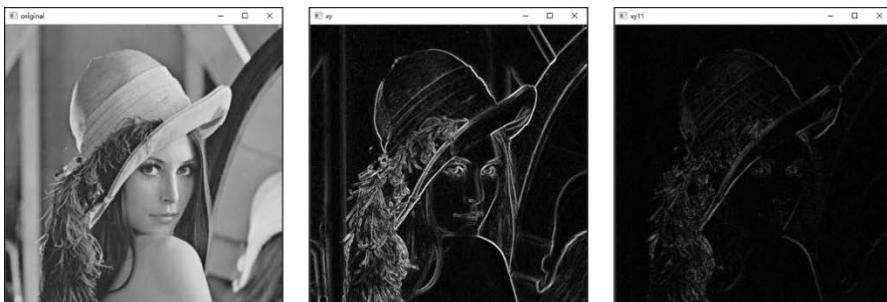


图 9-17 【例 9.7】程序的运行结果

9.3 Scharr 算子及函数使用

在离散的空间上，有很多方法可以用来计算近似导数，在使用 3×3 的 Sobel 算子时，可能计算结果并不太精准。OpenCV 提供了 Scharr 算子，该算子具有和 Sobel 算子同样的速度，且精度更高。可以将 Scharr 算子看作对 Sobel 算子的改进，其核通常为：

$$G_x = \begin{bmatrix} -3 & 0 & 3 \\ -10 & 0 & 10 \\ -3 & 0 & 3 \end{bmatrix}$$

$$G_y = \begin{bmatrix} -3 & -10 & -3 \\ 0 & 0 & 10 \\ 3 & 10 & 3 \end{bmatrix}$$

OpenCV 提供了函数 cv2.Scharr() 来计算 Scharr 算子，其语法格式如下：

```
dst = cv2.Scharr( src, ddepth, dx, dy[, scale[, delta[, borderType]]] )
```

式中:

- `dst` 代表输出图像。
- `src` 代表原始图像。
- `ddepth` 代表输出图像深度。该值与函数 `cv2.Sobel()` 中的参数 `ddepth` 的含义相同, 具体可以参考表 9-1。
- `dx` 代表 x 方向上的导数阶数。
- `dy` 代表 y 方向上的导数阶数。
- `scale` 代表计算导数值时的缩放因子, 该项是可选项, 默认值是 1, 表示没有缩放。
- `delta` 代表加到目标图像上的亮度值, 该项是可选项, 默认值为 0。
- `borderType` 代表边界样式。具体可以参考表 9-2。

在函数 `cv2.Sobel()` 中介绍过, 如果 `ksize=-1`, 则会使用 Scharr 滤波器。

因此, 如下语句:

```
dst=cv2.Scharr(src, ddepth, dx, dy)
```

和

```
dst=cv2.Sobel(src, ddepth, dx, dy, -1)
```

是等价的。

函数 `cv2.Scharr()` 和函数 `cv2.Sobel()` 的使用方式基本一致。

首先, 需要注意的是, 参数 `ddepth` 的值应该设置为 “`cv2.CV_64F`”, 并对函数 `cv2.Scharr()` 的计算结果取绝对值, 才能保证得到正确的处理结果。具体语句为:

```
dst=Scharr(src, cv2.CV_64F, dx, dy)
dst= cv2.convertScaleAbs(dst)
```

另外, 需要注意的是, 在函数 `cv2.Scharr()` 中, 要求参数 `dx` 和 `dy` 满足条件:

```
dx >= 0 && dy >= 0 && dx+dy == 1
```

因此, 参数 `dx` 和参数 `dy` 的组合形式有:

- 计算 x 方向边缘 (梯度): `dx=1, dy=0`。
- 计算 y 方向边缘 (梯度): `dx=0, dy=1`。
- 计算 x 方向与 y 方向的边缘叠加: 通过组合方式实现。

下面分别对上述情况进行简要说明。

1. 计算 x 方向边缘 (梯度): `dx=1, dy=0`

此时, 使用的语句是:

```
dst=Scharr(src, ddpeth, dx=1, dy=0)
```

2. 计算 y 方向边缘 (梯度): `dx=0, dy=1`

此时, 使用的语句是:

```
dst=Scharr(src, ddpeth, dx=0, dy=1)
```



3. 计算x方向与y方向的边缘叠加

将两个方向的边缘相加，使用的语句是：

```
dx=Scharr(src, ddpeth, dx=1, dy=0)
dy=Scharr(src, ddpeth, dx=0, dy=1)
Scharrxy=cv2.addWeighted(dx,0.5,dy,0.5,0)
```

需要注意的是，参数 dx 和 dy 的值不能都为 1。例如，如下语句是错误的：

```
dst=Scharr(src, ddpeth, dx=1, dy=1)
```

【例 9.8】 使用函数 `cv2.Scharr()` 获取图像水平方向的边缘信息。

根据题目要求，设计程序如下：

```
import cv2
o = cv2.imread('Scharr.bmp',cv2.IMREAD_GRAYSCALE)
Scharrx = cv2.Scharr(o,cv2.CV_64F,1,0)
Scharrx = cv2.convertScaleAbs(Scharrx)
cv2.imshow("original",o)
cv2.imshow("x",Scharrx)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行程序，结果如图 9-18 所示。

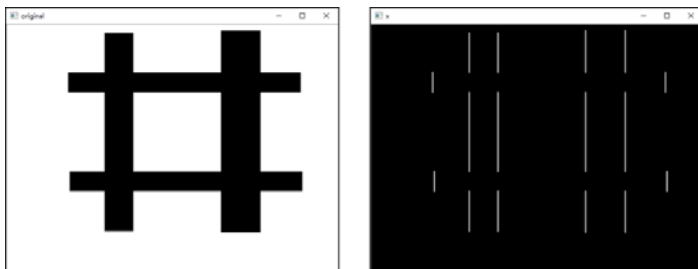


图 9-18 计算水平方向的边缘信息

【例 9.9】 使用函数 `cv2.Scharr()` 获取图像垂直方向的边缘信息。

根据题目要求，设计程序如下：

```
import cv2
o = cv2.imread('Scharr.bmp',cv2.IMREAD_GRAYSCALE)
Scharry = cv2.Scharr(o,cv2.CV_64F,0,1)
Scharry = cv2.convertScaleAbs(Scharry)
cv2.imshow("original",o)
cv2.imshow("y",Scharry)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行程序，结果如图 9-19 所示。

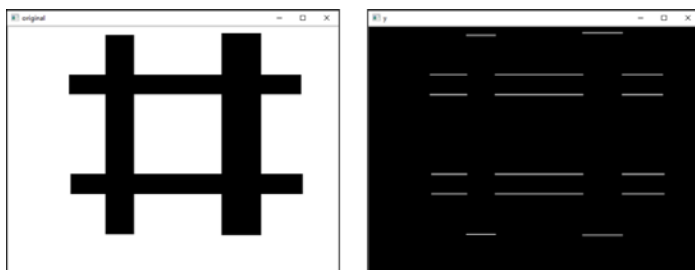


图 9-19 计算垂直方向的边缘信息

【例 9.10】使用函数 `cv2.Scharr()` 实现水平方向和垂直方向边缘叠加的效果。

根据题目要求，设计程序如下：

```
import cv2
o = cv2.imread('scharr.bmp', cv2.IMREAD_GRAYSCALE)
scharrx = cv2.Scharr(o, cv2.CV_64F, 1, 0)
scharry = cv2.Scharr(o, cv2.CV_64F, 0, 1)
scharrx = cv2.convertScaleAbs(scharrx)
scharry = cv2.convertScaleAbs(scharry)
scharxy = cv2.addWeighted(scharrx, 0.5, scharry, 0.5, 0)
cv2.imshow("original", o)
cv2.imshow("xy", scharxy)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行程序，结果如图 9-20 所示。

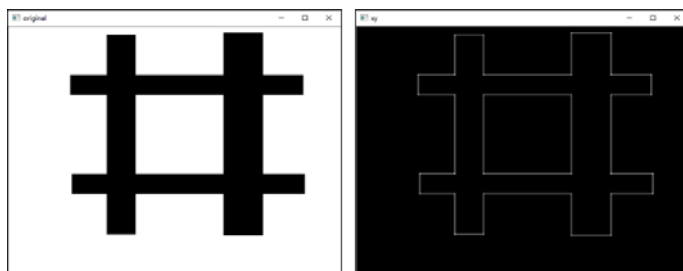


图 9-20 水平方向和垂直方向边缘的叠加

【例 9.11】观察将函数 `cv2.Scharr()` 的参数 `dx`、`dy` 同时设置为 1 时，程序的运行情况。

根据题目要求，设计程序如下：

```
import cv2
import numpy as np
o = cv2.imread('image\\Scharr.bmp', cv2.IMREAD_GRAYSCALE)
Scharrxy11 = cv2.Scharr(o, cv2.CV_64F, 1, 1)
cv2.imshow("original", o)
cv2.imshow("xy11", Scharrxy11)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行程序，报错如下：



```
error: OpenCV(3.4.2) C:\Miniconda3\conda-bld\opencv-suite_1534379934306\work\
modules\imgproc\src\deriv.cpp:67: error: (-215:Assertion failed) dx >= 0 && dy >=
0 && dx+dy == 1 in function 'cv::getScharrKernels'
```

【分析】

需要注意，不允许将函数 `cv2.Scharr()` 的参数 `dx` 和 `dy` 的值同时设置为 1。因此，本例中将这两个参数的值都设置为 1 后，程序会报错。

【例 9.12】使用函数 `cv2.Sobel()` 完成 Scharr 算子的运算。

当函数 `cv2.Sobel()` 中 `ksize` 的参数值为 -1 时，就会使用 Scharr 算子进行运算。因此，

```
dst=cv2.Sobel(src, ddpeth, dx, dy, -1)
```

等价于

```
dst=cv2.Scharr(src, ddpeth, dx, dy)
```

根据题目要求及分析，设计程序如下：

```
import cv2
o = cv2.imread('Sobel4.bmp', cv2.IMREAD_GRAYSCALE)
Scharrx = cv2.Sobel(o, cv2.CV_64F, 1, 0, -1)
Scharry = cv2.Sobel(o, cv2.CV_64F, 0, 1, -1)
Scharrx = cv2.convertScaleAbs(Scharrx)
Scharry = cv2.convertScaleAbs(Scharry)
cv2.imshow("original", o)
cv2.imshow("x", Scharrx)
cv2.imshow("y", Scharry)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行程序，结果如图 9-21 所示，其中左图为原始图像，中间的图为水平边缘图像，右图为垂直边缘图像。

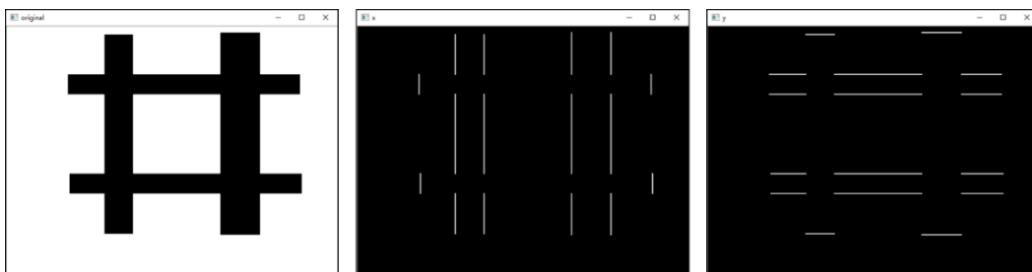


图 9-21 【例 9.12】程序的运行结果

9.4 Sobel 算子和 Scharr 算子的比较

Sobel 算子的缺点是，当其核结构较小时，精确度不高，而 Scharr 算子具有更高的精度。Sobel 算子和 Scharr 算子的核结构如图 9-22 所示。

Sobel算子

-1	0	1
-2	0	2
-1	0	1

-1	-2	-1
0	0	0
1	2	1

Scharr算子

-3	0	3
-10	0	10
-3	0	3

-3	-10	-3
0	0	0
3	10	3

图 9-22 Sobel 算子和 Scharr 算子的核结构

【例 9.13】 分别使用 Sobel 算子和 Scharr 算子计算一幅图像的水平边缘和垂直边缘的叠加信息。

根据题目要求，设计程序如下：

```
import cv2
o = cv2.imread('lena.bmp',cv2.IMREAD_GRAYSCALE)
Sobelx = cv2.Sobel(o,cv2.CV_64F,1,0,ksize=3)
Sobely = cv2.Sobel(o,cv2.CV_64F,0,1,ksize=3)
Sobelx = cv2.convertScaleAbs(Sobelx)
Sobely = cv2.convertScaleAbs(Sobely)
Sobelxy = cv2.addWeighted(Sobelx,0.5,Sobely,0.5,0)
Scharrx = cv2.Scharr(o,cv2.CV_64F,1,0)
Scharry = cv2.Scharr(o,cv2.CV_64F,0,1)
Scharrx = cv2.convertScaleAbs(Scharrx)
Scharry = cv2.convertScaleAbs(Scharry)
Scharrxxy = cv2.addWeighted(Scharrx,0.5,Scharry,0.5,0)
cv2.imshow("original",o)
cv2.imshow("Sobelxy",Sobelxy)
cv2.imshow("Scharrxxy",Scharrxxy)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行程序，结果如图 9-23 所示。



图 9-23 【例 9.13】程序的运行结果

9.5 Laplacian 算子及函数使用

Laplacian（拉普拉斯）算子是一种二阶导数算子，其具有旋转不变性，可以满足不同方向的图像边缘锐化（边缘检测）的要求。通常情况下，其算子的系数之和需要为零。例如，一个 3×3 大小的 Laplacian 算子如图 9-24 所示。

0	1	0
1	-4	1
0	1	0

图 9-24 3×3 大小的 Laplacian 算子

Laplacian 算子类似二阶 Sobel 导数，需要计算两个方向的梯度值。例如，在图 9-25 中：

- 左图是 Laplacian 算子。
- 右图是一个简单图像，其中有 9 个像素点。

0	1	0
1	-4	1
0	1	0

P1	P2	P3
P4	P5	P6
P7	P8	P9

图 9-25 Laplacian 算子示例

计算像素点 P5 的近似导数值，如下：

$$P5_{lap} = (P2 + P4 + P6 + P8) - 4 \cdot P5$$

图 9-26 展示了像素点与周围点的一些实例，其中：

- 在左图中，像素点 P5 与周围像素点的值相差较小，得到的计算结果值较小，边缘不明显。
- 在中间的图中，像素点 P5 与周围像素点的值相差较大，得到的计算结果值较大，边缘较明显。
- 在右图中，像素点 P5 与周围像素点的值相差较大，得到的计算结果值较大，边缘较明显。

	94	
80	88	92
	85	

非边界（梯度值小）

$$P5_{lap} = (94 + 80 + 92 + 85) - 4 \times 88$$

	200	
204	88	175
	158	

边界（梯度值大）

$$P5_{lap} = (200 + 204 + 175 + 158) - 4 \times 88$$

	20	
24	88	17
	15	

边界（梯度值大）

$$P5_{lap} = (20 + 24 + 17 + 15) - 4 \times 88$$

图 9-26 Laplacian 算子计算示例

需要注意，在上述图像中，计算结果的值可能为正数，也可能为负数。所以，需要对计算结果取绝对值，以保证后续运算和显示都是正确的。

在 OpenCV 内使用函数 `cv2.Laplacian()` 实现 Laplacian 算子的计算，该函数的语法格式为：
`dst = cv2.Laplacian( src, ddepth[, ksize[, scale[, delta[, borderType]]]] )`

式中：

- `dst` 代表目标图像。
- `src` 代表原始图像。
- `ddepth` 代表目标图像的深度。
- `ksize` 代表用于计算二阶导数的核尺寸大小。该值必须是正的奇数。
- `scale` 代表计算 Laplacian 值的缩放比例因子，该参数是可选的。默认情况下，该值为 1，表示不进行缩放。
- `delta` 代表加到目标图像上的可选值，默认为 0。
- `borderType` 代表边界样式。

该函数分别对 x 、 y 方向进行二次求导，具体为：

$$dst = \Delta src = \frac{\partial^2 src}{\partial x^2} + \frac{\partial^2 src}{\partial y^2}$$

上式是当 `ksize` 的值大于 1 时的情况。当 `ksize` 的值为 1 时，Laplacian 算子计算时采用的 3×3 的核如下：

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

通过从图像内减去它的 Laplacian 图像，可以增强图像的对比度，此时其算子如图 9-27 所示。

0	1	0
1	-5	1
0	1	0

图 9-27 扩展算子

【例 9.14】 使用函数 `cv2.Laplacian()` 计算图像的边缘信息。

根据题目要求，设计程序如下：

```
import cv2
o = cv2.imread('Laplacian.bmp',cv2.IMREAD_GRAYSCALE)
Laplacian = cv2.Laplacian(o,cv2.CV_64F)
Laplacian = cv2.convertScaleAbs(Laplacian)
cv2.imshow("original",o)
cv2.imshow("Laplacian",Laplacian)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行程序，结果如图 9-28 所示，其中左图为原始图像，右图为得到的边缘信息。

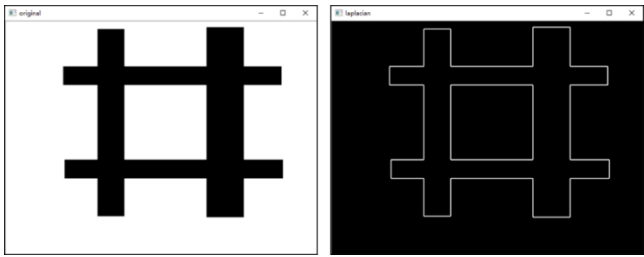


图 9-28 【例 9.14】程序的运行结果

9.6 算子总结

Sobel 算子、Scharr 算子、Laplacian 算子都可以用作边缘检测，它们的核如图 9-29 所示。

Sobel算子					
-1	0	1	-1	-2	-1
-2	0	2	0	0	0
-1	0	1	1	2	1
Laplacian算子					
Scharr算子					
-3	0	3	-3	-10	-3
-10	0	10	0	0	0
-1	0	3	3	10	3
0	1	0	1	-4	1
1	-4	1	0	1	0

图 9-29 算子核

Sobel 算子和 Scharr 算子计算的都是一阶近似导数的值。通常情况下，可以将它们表示为：

Sobel 算子= |左-右| / |下-上|

Scharr 算子= |左-右| / |下-上|

式中“|左-右|”表示左侧像素值减右侧像素值的结果的绝对值，“|下-上|”表示下方像素值减上方像素值的结果的绝对值。

Laplacian 算子计算的是二阶近似导数值，可以将它表示为：

Laplacian 算子= |左-右| + |左-右| + |下-上| + |下-上|

通过公式可以发现，Sobel 算子和 Scharr 算子各计算了一次“|左-右|”和“|下-上|”的值，而 Laplacian 算子分别计算了两次“|左-右|”和“|下-上|”的值。